Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 10-12

Applications of Morphological Processing: Boundaries, Region Filling, and Connected Components

Course Code: ECE 4224

Course Title: Digital Image Processing Sessional

Submitted to:
Oishi Jyoti
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 7, 2026

Contents

1 Experiment 10: Boundary Extraction **1**
- 1.1 Theory and Introduction . . . 1
- 1.2 Methodology . . . 1
  - 1.2.1 Matrix Definition . . . 1
  - 1.2.2 Python Implementation . . . 1
- 1.3 Results . . . 2
- 1.4 Discussion . . . 2
- 1.5 Conclusion . . . 2

2 Experiment 11: Region Filling **3**
- 2.1 Theory and Introduction . . . 3
- 2.2 Methodology . . . 3
  - 2.2.1 Matrix Definition . . . 3
  - 2.2.2 Python Implementation . . . 3
- 2.3 Results . . . 4
- 2.4 Discussion . . . 5
- 2.5 Conclusion . . . 5

3 Experiment 12: Connected Components Extraction **6**
- 3.1 Theory and Introduction . . . 6
- 3.2 Methodology . . . 6
  - 3.2.1 Matrix Definition . . . 6
  - 3.2.2 Python Implementation . . . 6
- 3.3 Results . . . 7
- 3.4 Discussion . . . 8
- 3.5 Conclusion . . . 8

References **9**

1 Experiment 10: Boundary Extraction

1.1 Theory and Introduction

Boundary extraction is a fundamental morphological operation used to identify the perimeter or outline of a binary object. The boundary of a set A, denoted as $\beta(A)$, can be obtained by first eroding A with a suitable structuring element B, and then performing a set difference between the original set A and its erosion [1].

$$\beta(A) = A - (A \ominus B)$$

Where $\ominus$ denotes morphological erosion. By eroding the image with a 3×3 structuring element of 1s, the outermost layer of pixels of the object is stripped away. Subtracting this shrunken object from the original leaves only that outermost layer.

1.2 Methodology

A binary image containing a solid square shape was defined, and the boundary was extracted using morphological erosion and matrix subtraction.

1.2.1 Matrix Definition

A 6×6 binary image matrix A was created containing a 4×4 solid block of 1s. A 3×3 Structuring Element (SE) B consisting entirely of 1s was utilized to ensure a uniform 1-pixel thick boundary extraction.

1.2.2 Python Implementation

The SciPy $binary_erosion$ function was utilized to compute $A \ominus B$. The resulting eroded matrix was then subtracted from the original matrix A. The matrices were cast to integer types to perform the arithmetic subtraction $A - C$.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import binary_erosion
import os

os.makedirs("./images/output", exist_ok=True)

# Step 1: Define the binary image A
A = np.array(
    [
        [0, 0, 0, 0, 0, 0],
        [0, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0],
        [0, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0],
    ],
    dtype=bool,
)

# Step 2 & 3: Define Structuring Element B (3x3 ones)
B = np.ones((3, 3), dtype=bool)

# Step 4: Perform erosion
C = binary_erosion(A, structure=B)

# Step 5: Boundary extraction
# Subtracting eroded image from original
boundary = A.astype(int) - C.astype(int)

# Step 6: Display results
fig, axes = plt.subplots(1, 3, figsize=(17, 5))


def plot_matrix(ax, matrix, title):
    ax.imshow(matrix, cmap="gray", vmin=0, vmax=1)
    ax.set_title(title)
    # Add grid lines for clarity
    h, w = matrix.shape
    ax.set_xticks(np.arange(-0.5, w, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, h, 1), minor=True)
    ax.grid(which="minor", color="gray", linestyle="-", linewidth=1)
    ax.tick_params(which="minor", size=0)

    # Annotate values
    for i in range(h):
        for j in range(w):
```

```python
            val = int(matrix[i, j])
            color = "white" if val == 0 else "black"
            ax.text(j, i, str(val), ha="center", va="center", color=color, fontsize=10)


plot_matrix(axes[0], A, "Original Image A")
plot_matrix(axes[1], C, r"Eroded Image (A $\ominus$ B)")
plot_matrix(axes[2], boundary, r"Boundary = A - (A $\ominus$ B)")

plt.tight_layout()
plt.savefig("./images/output/lab10_boundary.png")
print("Saved lab10_boundary.png to output folder.")
# plt.show()
```

1.3 Results

The code successfully isolated the perimeter of the binary object.

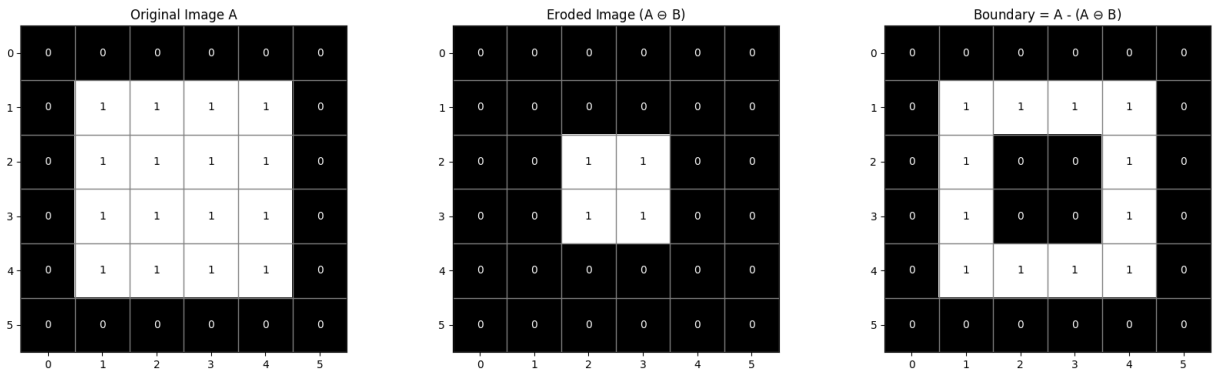

Figure 1: Boundary Extraction Process. Left: Original 4×4 object. Center: The object after erosion, reduced to a 2×2 core. Right: The extracted boundary, representing the difference between the first two matrices.

1.4 Discussion

The mathematical set difference proved effective for outline generation. The original object extended from index (1,1) to (4,4). After erosion with the 3×3 structuring element, only the inner core from index (2,2) to (3,3) survived, as these were the only pixels where the structuring element fit completely inside the object. Subtracting this 2×2 core from the original 4×4 object perfectly isolated the 1-pixel thick outer ring.

1.5 Conclusion

The mechanism of morphological boundary extraction was successfully validated. It was demonstrated that combining non-linear erosion with simple arithmetic subtraction is a robust method for isolating the perimeters of solid binary shapes.

2 Experiment 11: Region Filling

2.1 Theory and Introduction

Region filling is a morphological algorithm used to fill holes in binary images. Given an image A containing boundaries, and a point p inside a boundary, the goal is to fill the entire enclosed region with 1s. This is achieved iteratively by dilating a seed point and constraining its growth so it does not spill past the boundary. The recursive equation is [1]:

$$X_k = (X_{k-1} \oplus B) \cap A^c$$

Where X_0 is the initial seed point, B is a cross-shaped structuring element, and A^c is the complement of the boundary image. The iteration terminates when $X_k = X_{k-1}$. The final filled image is the union of the original boundary and the filled region $(A \cup X_k)$.

2.2 Methodology

A hollow square boundary was defined, an internal seed point was selected, and the iterative morphological filling algorithm was implemented.

2.2.1 Matrix Definition

A 7×7 binary image A was defined containing a continuous 1-pixel thick boundary outline. A cross-shaped 3×3 Structuring Element B was used to prevent diagonal spilling. An initial seed matrix X_0 was created with a single 1 at coordinate (3,3), strictly inside the boundary.

2.2.2 Python Implementation

A $whileTrue$ loop was constructed to execute the recursive equation. In each iteration, the current region X_{prev} was dilated using $binary_dilation$ and immediately intersected (logical AND) with A^c. The loop was broken when $np.array_equal$ confirmed the region had stopped growing.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import binary_dilation
import os

os.makedirs("./images/output", exist_ok=True)

# Step 1: Define boundary image A
A = np.array(
    [
        [0, 0, 0, 0, 0, 0, 0],
        [0, 1, 1, 1, 1, 1, 0],
        [0, 1, 0, 0, 0, 1, 0],
        [0, 1, 0, 0, 0, 1, 0],
        [0, 1, 0, 0, 0, 1, 0],
        [0, 1, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0],
    ],
    dtype=bool,
)

# Step 2: Complement of A
Ac = ~A

# Step 3: Structuring Element (cross shape)
B = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=bool)

# Step 4: Seed point
# MATLAB coordinates (4,4) map to Python 0-based index [3, 3]
X_prev = np.zeros_like(A)
X_prev[3, 3] = 1

# Save initial seed state for visualization later
X0 = X_prev.copy()

# Step 5: Iteration (Morphological Region Filling Algorithm)
while True:
    # Dilate the previous region
    X_dilated = binary_dilation(X_prev, structure=B)

    # Intersect with the complement of the boundary
```

```python
    X_new = X_dilated & Ac

    # Check for convergence (if the region stops growing)
    if np.array_equal(X_new, X_prev):
        break

    X_prev = X_new.copy()

Xk = X_new

# Step 6: Final result
# Union of original boundary and the filled region
Filled = A | Xk

# Step 7: Display Setup
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()


# Custom Plotting Function matching your MATLAB style
def showMatrix(ax, mat, titleText):
    ax.imshow(mat, cmap="gray", vmin=0, vmax=1)
    ax.set_title(titleText, fontsize=14)

    # Set up grid lines on pixel borders
    h, w = mat.shape
    ax.set_xticks(np.arange(-0.5, w, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, h, 1), minor=True)
    ax.grid(which="minor", color="gray", linestyle="-", linewidth=1)
    ax.tick_params(which="minor", size=0)

    # Hide major ticks
    ax.set_xticks([])
    ax.set_yticks([])

    # Add red numbers to the center of each cell
    for i in range(h):
        for j in range(w):
            val = int(mat[i, j])
            ax.text(
                j,
                i,
                str(val),
                ha="center",
                va="center",
                color="red",
                fontweight="bold",
                fontsize=12,
            )


# Plot all stages
showMatrix(axes[0], A, "Original Boundary A")
showMatrix(axes[1], Ac, "Complement $A^c$")
showMatrix(axes[2], X0, "Initial Seed $X_0$")
showMatrix(axes[3], Xk, "Filled Region $X_k$")
showMatrix(axes[4], Filled, "Final Filled Image")
showMatrix(axes[5], B, "Structuring Element B")

plt.tight_layout()
plt.savefig("./images/output/lab11_fill.png")
print("Saved lab11_fill.png to output folder.")
# plt.show()
```

2.3 Results

The iterative algorithm successfully flooded the internal region without breaching the boundary walls.

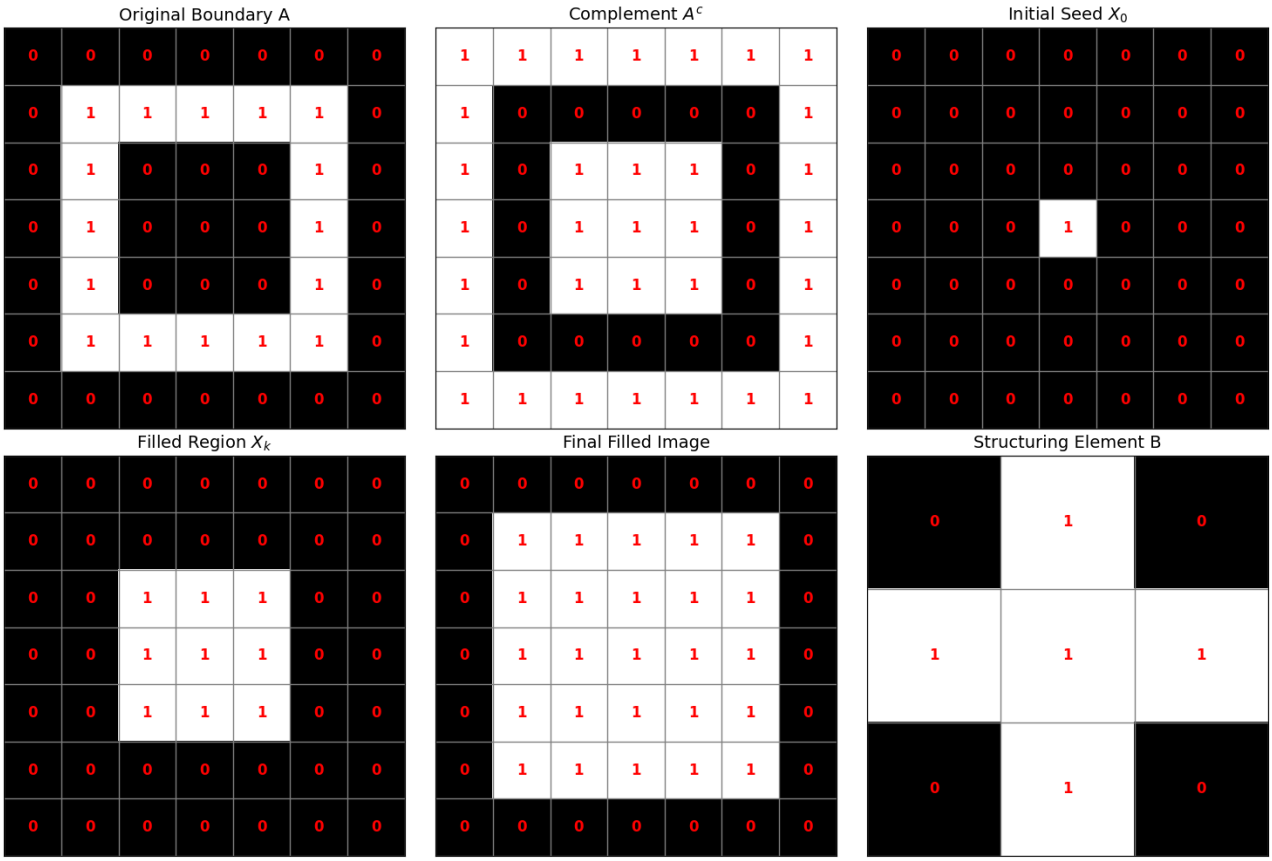

Figure 2: Region Filling Process. The initial seed X_0 is iteratively dilated and constrained by A^c until it fills the inner void. The final image is the union of A and X_k.

2.4 Discussion

The importance of the A^c constraint was clearly demonstrated. The cross-shaped structuring element B caused the seed point to grow horizontally and vertically. Without the intersection with A^c, the dilation would have eventually filled the entire 7×7 matrix. By continuously applying A^c, any dilated pixels that overlapped with the original boundary (where $A^c = 0$) were immediately erased, effectively using the boundary as an impassable wall. The process naturally converged when the entire hollow space was occupied.

2.5 Conclusion

The theoretical algorithm for morphological region filling was successfully implemented. It was verified that constrained iterative dilation from a known internal seed point is a reliable method for filling enclosed spaces within binary boundaries.

3 Experiment 12: Connected Components Extraction

3.1 Theory and Introduction

Extracting connected components is essential for automated image analysis, object counting, and classification. The algorithm isolates a complete, contiguous object from a complex binary image given a single known point on that object. The recursive equation is mathematically similar to region filling [1]:

$$X_k = (X_{k-1} \oplus B) \cap A$$

Where X_0 contains the initial point, B is the structuring element determining connectivity (4-connected or 8-connected), and A is the original image. Unlike region filling which intersects with the complement, this algorithm intersects with the original image A to constrain the growth strictly to existing foreground pixels. Iteration stops when $X_k = X_{k-1}$.

3.2 Methodology

A binary matrix with multiple separate objects was defined. A seed point was placed on one specific object, and the iterative extraction algorithm was applied.

3.2.1 Matrix Definition

A 7×7 binary image A was defined containing an irregular L-shaped object and a separate, isolated point. An 8-connected Structuring Element B (a 3×3 matrix of all 1s) was utilized. A seed point was placed at coordinate (2,3), which lies on the L-shaped object.

3.2.2 Python Implementation

The implementation mirrors the region-filling loop. The $binary_dilation$ function was applied to the seed matrix, and the result was intersected (logical AND) with the original image A. The loop terminated upon convergence.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import binary_dilation
import os

os.makedirs("./images/output", exist_ok=True)

# Step 1: Define binary image A
A = np.array(
    [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 1, 1, 0, 1, 1, 1, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 1, 1, 1, 0, 0],
        [0, 0, 0, 0, 0, 1, 0, 1, 0, 0],
        [0, 0, 0, 0, 0, 1, 1, 1, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 1, 1, 0, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 1, 1, 0],
    ],
    dtype=bool,
)

# Step 2: Structuring Element (8-connected block)
# Note: The MATLAB comment says "4-connected" but the matrix is a 3x3 of all 1s (which is 8-connected).
B = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]], dtype=bool)

# Step 3: Select a seed point from a component
# MATLAB coordinates (3,4) map to Python 0-based index [1,1]
X_prev = np.zeros_like(A)
X_prev[3, 8] = 1

# Save initial seed state for visualization
X0 = X_prev.copy()

# Step 4: Iterative extraction (Connected Component Algorithm)
while True:
    # Dilate the previous region
    X_dilated = binary_dilation(X_prev, structure=B)
```

```python

    # Intersect with the original image to constrain growth to foreground pixels
    X_new = X_dilated & A

    # Check for convergence
    if np.array_equal(X_new, X_prev):
        break

    X_prev = X_new.copy()

Xk = X_new  # Final connected component

# -------- Display Setup --------
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
axes = axes.flatten()


# Custom Plotting Function matching your style
def showMatrix(ax, mat, titleText):
    ax.imshow(mat, cmap="gray", vmin=0, vmax=1)
    ax.set_title(titleText, fontsize=14)

    # Set up grid lines on pixel borders
    h, w = mat.shape
    ax.set_xticks(np.arange(-0.5, w, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, h, 1), minor=True)
    ax.grid(which="minor", color="gray", linestyle="-", linewidth=1)
    ax.tick_params(which="minor", size=0)

    # Hide major ticks
    ax.set_xticks([])
    ax.set_yticks([])

    # Add red numbers to the center of each cell
    for i in range(h):
        for j in range(w):
            val = int(mat[i, j])
            ax.text(
                j,
                i,
                str(val),
                ha="center",
                va="center",
                color="red",
                fontweight="bold",
                fontsize=12,
            )


# Plot all stages
showMatrix(axes[0], A, "Original Image A")
showMatrix(axes[1], X0, "Initial Seed $X_0$")
showMatrix(axes[2], Xk, "Extracted Connected Component $X_k$")
showMatrix(axes[3], B, "Structuring Element B")

plt.tight_layout()
plt.savefig("./images/output/lab12_connected.png")
print("Saved lab12_connected.png to output folder.")
# plt.show()
```

3.3 Results

The algorithm successfully isolated the contiguous L-shaped object while ignoring the disconnected noise point.

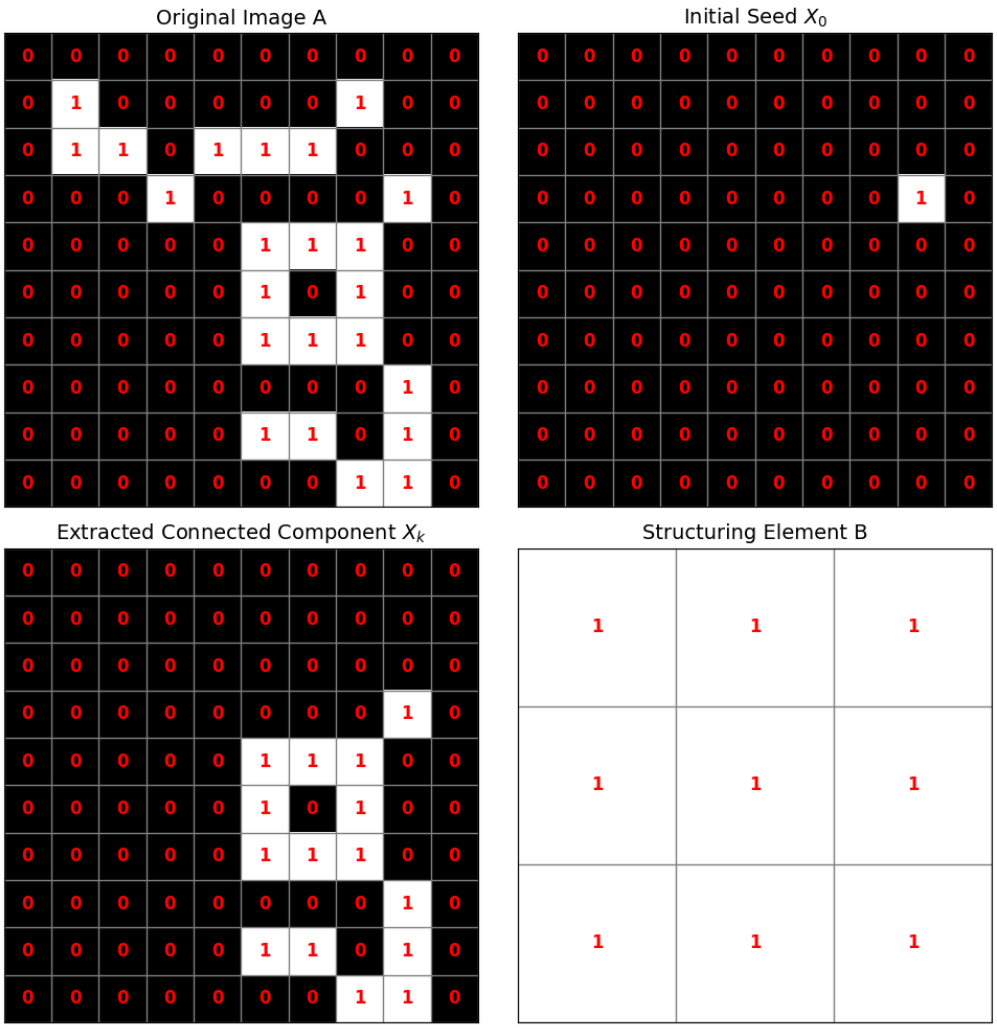

Figure 3: Connected Components Extraction. The seed X_0 iteratively grows along the foreground pixels of A. The isolated point at (1,5) in the original image is successfully ignored.

3.4 Discussion

The distinction between the intersection constraints was highlighted. By applying the logical AND operation with the original image A, the dilated seed was only allowed to survive where foreground pixels already existed. Over successive iterations, the seed propagated through the entire L-shaped structure. Because the structuring element B could not bridge the gap of zeros between the L-shape and the isolated point at (1,5), that point was never reached by the dilation process and was effectively filtered out of the final component X_k.

3.5 Conclusion

The morphological extraction of connected components was successfully validated. It was demonstrated that iterative dilation constrained by the original image bounds is an effective method for isolating specific contiguous objects from multi-object binary images.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Boston, MA, USA: Pearson, 2018.